



**ICG:**

# Shaders e Iluminação (GLSL)

---

Prof<sup>a</sup>. Tainá Isabela



# O Pipeline Gráfico Tradicional vs. Programável

Nas versões antigas do OpenGL, a renderização utilizava o pipeline de função fixa, onde as operações de geometria e iluminação eram predefinidas e imutáveis.

O pipeline programável moderno substitui essa estrutura rígida por estágios flexíveis controlados diretamente pelo desenvolvedor. Essa evolução permite o controle total sobre a manipulação de vértices e o cálculo de cor de cada pixel na tela.

# Visão Geral dos Estágios do Pipeline

O pipeline gráfico sequencia uma série de etapas que transformam dados tridimensionais em uma imagem bidimensional na tela. Os dados de entrada, compostos por vértices e atributos, passam por transformações geométricas, montagem de primitivas e rasterização.

Entre esses estágios, destacam-se o Vertex Shader e o Fragment Shader como os blocos fundamentais de programação customizada.

# O papel do Vertex Shader

O Vertex Shader atua como o primeiro estágio programável do pipeline, processando cada vértice de forma isolada e paralela. Suas principais atribuições envolvem a aplicação de matrizes de transformação geométrica (Modelo, Visão e Projeção) para posicionar os objetos no espaço tridimensional.

Além disso, este shader prepara e encaminha atributos como normais e coordenadas de textura para os estágios seguintes.

# Rasterização

Após o processamento dos vértices e a montagem das primitivas geométricas, o hardware executa a rasterização. Este processo consiste em converter os triângulos tridimensionais em fragmentos, que representam os potenciais pixels da imagem final.

Durante a rasterização, os atributos calculados nos vértices são interpolados linearmente ao longo da superfície da primitiva antes de chegarem ao próximo shader.

# Fragment Shader

O Fragment Shader é o estágio encarregado de determinar a cor final de cada fragmento gerado pela rasterização. É neste bloco de código que se implementam os cálculos mais complexos de iluminação, mapeamento de texturas, sombras e efeitos pós-processamento.

A eficiência deste shader é crítica, dado que ele é executado milhões de vezes por segundo para preencher a tela do usuário.

# GLSL (OpenGL Shading Language)

A GLSL é uma linguagem de alto nível com sintaxe baseada na linguagem C, projetada especificamente para execução direta na GPU (Unidade de Processamento Gráfico).

Ela oferece suporte nativo a operações vetoriais e matriciais, fundamentais para a álgebra linear aplicada à computação gráfica. Os códigos escritos em GLSL são compilados e linkados em tempo de execução pela própria aplicação OpenGL.

# Tipos de Dados e Variáveis em GLSL

A linguagem GLSL introduz tipos de dados específicos para otimizar os cálculos gráficos na GPU. Além dos tipos primitivos como inteiros e pontos flutuantes, destacam-se os vetores (`vec2`, `vec3`, `vec4`) e as matrizes (`mat3`, `mat4`).

A manipulação desses tipos permite realizar operações complexas de translação, rotação e escala com poucas linhas de código.



# Qualificadores de Entrada e Saída: Uniforms

Os qualificadores de escopo gerenciam o fluxo de dados entre a CPU, a GPU e os diferentes estágios de shaders. As variáveis do tipo uniform possuem valor constante durante toda a execução de um comando de renderização.

Elas são enviadas pela aplicação na CPU para comunicar dados globais, tais como a posição das luzes e as matrizes de transformação.

# Qualificadores de Entrada e Saída: In e Out

As variáveis declaradas com o qualificador in representam os dados de entrada recebidos pelo shader, enquanto as declaradas com out definem os dados de saída. No Vertex Shader, os dados de entrada vêm dos buffers de vértices.

A saída do Vertex Shader conecta-se diretamente à entrada do Fragment Shader através da interpolação automática feita pelo hardware.

# Fundamentos da Iluminação Óptica Computacional

A simulação de iluminação em tempo real na computação gráfica baseia-se em aproximações matemáticas dos fenômenos físicos da luz. Em vez de rastrear fótons individuais, os modelos padrão calculam a interação dos raios luminosos com os vetores normais das superfícies.

O objetivo reside em equilibrar o realismo visual com o desempenho computacional exigido pelas aplicações interativas.

# O Modelo de Iluminação de Phong

O modelo de Phong é uma das aproximações mais difundidas para a renderização de superfícies iluminadas em tempo real. Este modelo empírico divide a iluminação final em três componentes independentes e cumulativas: a luz ambiente, a reflexão difusa e a reflexão especular.

A combinação matemática dessas três parcelas determina a intensidade e a cor resultantes no fragmento.

# Componente de Iluminação Ambiente

A iluminação ambiente simula a luz que foi refletida tantas vezes pelo cenário que se espalhou de maneira uniforme pelo ambiente. Matematicamente, trata-se de um valor constante que afeta todos os objetos por igual, independentemente de suas posições ou orientações.

Essa componente garante que as partes não iluminadas diretamente por uma fonte de luz não fiquem completamente pretas.

# Componente de Iluminação Difusa

A iluminação difusa modela o espalhamento direcionado da luz quando esta atinge uma superfície rugosa ou opaca, refletindo-se em todas as direções de forma igual.

O cálculo baseia-se na Lei do Cosseno de Lambert, que utiliza o produto escalar entre o vetor normal da superfície e o vetor de direção da luz. Quanto mais perpendicular for o raio luminoso em relação à superfície, maior será a intensidade da cor.

# Vetores Normais

Os vetores normais são perpendiculares à superfície e fundamentais para os cálculos de física óptica no pipeline. Eles definem a orientação espacial de cada face ou vértice, permitindo determinar o ângulo exato de incidência dos raios de luz.

Sem a correta manipulação e transformação das normais no Vertex Shader, torna-se impossível calcular os efeitos de sombreamento difuso e especular.

# Componente de Iluminação Especular

A componente especular simula o brilho característico de superfícies polidas ou metálicas, onde a reflexão da luz ocorre em uma direção preferencial.

O cálculo depende diretamente da posição da câmera do observador em relação ao objeto. A intensidade do brilho é determinada pelo ângulo entre o vetor de reflexão do raio de luz e o vetor que aponta para o visualizador.



# O Fator de Brilho (Shininess)

No cálculo especular de Phong, a variável de brilho (ou expoente de reflexão) controla a concentração do ponto de luz reflexivo na superfície. Um valor de brilho baixo resulta em uma mancha especular espalhada e suave, típica de materiais levemente foscos.

Valores elevados geram um ponto reflexivo pequeno e nítido, simulando materiais altamente polidos como vidro ou metal.

# Modelos de Sombreamento: Flat Shading

O sombreamento constante, ou Flat Shading, calcula a iluminação apenas uma vez por primitiva (triângulo), aplicando a cor resultante a toda a face. Este modelo ignora as variações internas de curvatura, resultando em uma aparência facetada onde as bordas dos polígonos ficam visíveis.

Embora seja computacionalmente muito barato, falha em representar superfícies orgânicas ou suaves.

# Modelos de Sombreamento: Gouraud Shading

O sombreamento de Gouraud realiza os cálculos matemáticos de iluminação nos vértices do modelo tridimensional dentro do Vertex Shader. As cores resultantes nos vértices são então interpoladas linearmente pela placa gráfica ao longo da superfície do triângulo.

Embora produza superfícies de aparência suave, este modelo falha em renderizar pontos de brilho especular que caiam no centro das faces.

# Modelos de Sombreamento: Phong Shading

O sombreamento de Phong (ou sombreamento por fragmento) transfere os cálculos de iluminação para o Fragment Shader. Em vez de interpolar as cores, o hardware interpola os vetores normais ao longo da superfície.

A iluminação é recalculada pixel por pixel, permitindo a reprodução precisa de brilhos especulares e transições suaves de luz, consolidando-se como o padrão para renderização de qualidade.

# Questionário



# Exercícios

1. Considerando que os objetos da cena já possuem o cálculo de iluminação ativo, altere o Fragment Shader para criar um efeito de alerta. Utilize uma variável uniform de tempo enviada pelo Python para fazer a componente difusa da luz oscilar suavemente entre a intensidade original e o tom vermelho máximo, utilizando a função  $\sin()$ .
2. Utilizando a estrutura de iluminação já existente em seus objetos, crie uma variável uniform no Fragment Shader para controlar o expoente de reflexão especular (shininess). Configure a aplicação em Python para alterar esse valor dinamicamente via teclado, permitindo alternar visualmente o aspecto do objeto entre um material fosco e um metal polido.